Perfect — this is exactly the right way to execute it.

Below is your **FULL Codex prompt pack**, step-by-step, ready to paste directly into VS Code Codex.

You will run these **IN ORDER**, using:

- **Ask (Plan-style)** → first
- **Code (Edit-style)** → second

⚠️ Rule: **Never skip the Ask prompt before running Code**

🔷 STEP 1 — Add Context to Input

🧠 ASK PROMPT (paste first)

```
You are working in /mnt/data/audit.js.

Do NOT write code yet.

Task:
Inspect validateSingleInput() and identify exactly how input is currently
validated and returned.

Goal:
Add a normalized "context" object for future launch-readiness scoring.

Constraints:
- Do not break existing API contract
- Preserve current required fields: copy, copy_type, goal
- Add optional fields only
- Campaign mode must remain untouched
- Defaults must be safe if fields are missing

Return:
1. Exact lines/functions to modify
2. Minimal patch strategy
3. Downstream dependencies affected
4. Risks to avoid
5. Whether this step should be split further
```

⚙️ CODE PROMPT

```
Implement Step 1 in /mnt/data/audit.js.

Goal:
```

```
Extend validateSingleInput() to return a normalized context object.

Add optional fields:
- channel
- traffic_source
- audience_temperature
- awareness_level
- market_category
- offer_model
- price_point
- sales_cycle
- claim_sensitivity
- competitive_maturity
- brand_proof_available

Requirements:
- Preserve backward compatibility
- Keep required fields unchanged
- Add safe defaults
- Do not modify model prompts yet
- Do not affect campaign mode

After completion:
- Summarize the diff
- List any new dependencies created
```

🔷 STEP 2 — Upgrade detectSignals()

🧠 ASK PROMPT

```
You are working in /mnt/data/audit.js.

Do NOT write code yet.

Task:
Inspect detectSignals() and explain its current structure and outputs.

Goal:
Upgrade it into a richer heuristic engine.

Add new signal groups:
- claims
- proof_markers
- authority_markers
- comparative_markers
- guarantee_markers
```

```
- urgency_markers
- ai_pattern_markers
- distinctness_markers
- offer_stack_markers
- risk_markers

Constraints:
- Preserve existing structure
- Add only new nested fields
- Keep it lightweight (regex only)
- No external dependencies

Return:
1. Proposed new signal schema
2. Exact modifications needed
3. Any breaking risks
```

⚙️ CODE PROMPT

```
Implement the detectSignals() upgrade in /mnt/data/audit.js.

Requirements:
- Keep existing keys unchanged
- Add new nested signal groups
- Use regex/string heuristics only
- Keep runtime fast
- Do not modify model prompts yet

Include detection for:
- claims (best, better, only)
- authority (doctor, certified, study)
- testimonials (quotes, reviews)
- guarantees (refund, risk-free)
- urgency (now, today, limited)
- quantification (%, $, numbers)
- AI cadence (repetitive short lines)
- distinctness markers

After completion:
- Summarize new signal groups
```

🔷 STEP 3 — Add Evidence Classifier

🧠 ASK PROMPT

```
You are working in /mnt/data/audit.js.

Do NOT write code yet.

Task:
Design a new helper function classifyEvidence(text, signals).

Goal:
Create a deterministic proof classification system.

It must detect:
- testimonial proof
- quantified proof
- expert proof
- mechanism proof
- operational proof
- product validation
- authority proof
- missing proof

Return:
1. Function signature
2. Output schema
3. Where it should be called
4. How it integrates with existing packet flow
```

⚙️ CODE PROMPT

```
Implement classifyEvidence(text, signals) in /mnt/data/audit.js.

Requirements:
- Pure function (no model calls)
- Use heuristics only
- Return structured evidence object
- Integrate into buildSingleAuditPacket()

Do NOT:
- Modify schemas yet
- Remove existing logic

After completion:
- Explain how evidence is computed
```

🔷 STEP 4 — Add Subsystem Prompts

🧠 ASK PROMPT

```
You are working in /mnt/data/audit.js.

Do NOT write code yet.

Task:
Prepare for multi-engine auditing.

Add prompt constants for:
- persuasion
- proof strength
- skepticism
- claim exposure
- decision synthesis

Constraints:
- Keep existing SINGLE_AUDIT prompt
- Keep schemas minimal
- Optimize for structured JSON output

Return:
1. Proposed prompt structure
2. Minimal schema for each subsystem
```

⚙️ CODE PROMPT

```
Add subsystem prompt constants and schemas in /mnt/data/audit.js.

Add:
- PERSUASION_AUDIT_SYSTEM_PROMPT
- PROOF_STRENGTH_SYSTEM_PROMPT
- SKEPTICISM_ENGINE_SYSTEM_PROMPT
- CLAIM_EXPOSURE_SYSTEM_PROMPT
- DECISION_SYNTHESIS_SYSTEM_PROMPT

Requirements:
- Keep current system intact
- JSON-only outputs
- Minimal schemas

After completion:
- List all new prompts and schemas
```

🔷 STEP 5 — Add Sanitizers

🧠 ASK PROMPT

```
You are working in /mnt/data/audit.js.

Do NOT write code yet.

Task:
Design sanitizers for each subsystem output.

Follow style of existing sanitizers.

Return:
1. Required sanitization rules
2. Edge cases to protect against
```

⚙️ CODE PROMPT

```
Implement subsystem sanitizers:

- sanitizePersuasionShape
- sanitizeProofStrengthShape
- sanitizeSkepticismShape
- sanitizeClaimExposureShape
- sanitizeDecisionShape

Requirements:
- Clamp numeric values
- Sanitize enums
- Ensure booleans
- Provide fallbacks

Do not modify runtime yet

After:
- Summarize protections added
```

🔷 STEP 6 — Add Subsystem Runners

🧠 ASK PROMPT

```
You are working in /mnt/data/audit.js.
```

```
Do NOT write code yet.

Task:
Plan runtime helpers for subsystem execution.

Return:
1. Input packet structure
2. Runner function design
3. Integration points
```

⚙️ CODE PROMPT

```
Add runner functions:

- runPersuasionAudit
- runProofStrengthAudit
- runSkepticismAudit
- runClaimExposureAudit
- runDecisionSynthesis

Requirements:
- Use runJsonModel()
- Sanitize outputs immediately
- Do not replace existing flow yet

After:
- Summarize runner inputs
```

🔷 STEP 7 — Replace Single Audit Flow

🧠 ASK PROMPT

```
You are working in /mnt/data/audit.js.

Do NOT write code yet.

Task:
Plan replacement of runSingleAssetAudit().

New flow:
- persuasion
- proof
- skepticism
- claims
- synthesis
```

```
Constraints:
- Keep backward compatibility
- Preserve legacy fields

Return:
1. Migration strategy
2. Compatibility plan
3. Risks
```

⚙️ CODE PROMPT

```
Refactor runSingleAssetAudit() to multi-engine flow.

Requirements:
- Run all subsystems
- Combine outputs
- Preserve legacy fields
- Add new fields:
  - persuasion_scores
  - proof_strength_index
  - skepticism_engine
  - claim_exposure

After:
- Summarize compatibility decisions
```

🔷 STEP 8 — Add Verdict Resolver

🧠 ASK PROMPT

```
You are working in /mnt/data/audit.js.

Do NOT write code yet.

Task:
Design deterministic verdict logic.

Include gates:
- weak proof blocks scaling
- weak offer blocks scaling
- high claim risk blocks launch

Return:
1. Rule order
```

```
2. Conflict resolution logic
```

⚙️ CODE PROMPT

```
Implement resolveLaunchVerdict().

Requirements:
- No model-based verdict
- Ladder:
  No-Go → Hold → Conditional → Test → Go → Scale
- Apply gating rules

Return:
- verdict
- confidence
- safe_to_test
- safe_to_scale
- primary_blocker

After:
- List all rules implemented
```

🔷 STEP 9 — Legacy Adapter

🧠 ASK PROMPT

```
Plan adapter to map new output → old UI format.

Return:
1. Field mappings
2. Derived fields
```

⚙️ CODE PROMPT

```
Implement toLegacySingleAuditShape().

Requirements:
- Preserve UI compatibility
- Map new scores → old fields

After:
- Explain mapping logic
```

🔷 STEP 10 — Campaign Upgrade

🧠 ASK PROMPT

```
Plan campaign packet upgrade.

Return:
- new fields to include
```

⚙️ CODE PROMPT

```
Upgrade campaign summary packet.

Requirements:
- include verdict, scores, blockers
- keep payload small

After:
- summarize new campaign fields
```

🔷 STEP 11 — Cleanup

🧠 ASK PROMPT

```
Identify dead code from old audit system.
```

⚙️ CODE PROMPT

```
Remove unused single-pass audit code.

Requirements:
- do not break anything
- keep compatibility

After:
- confirm removals
```

🧠 FINAL EXECUTION RULE

Do NOT rush.

Run like this:

1. Ask → read carefully
2. Code → apply
3. Review diff
4. Test
5. Move to next

🔥 If you follow this exactly

You will end up with:

- A **multi-engine audit system**
- Deterministic verdicts
- AI-resistant scoring
- True **pre-traffic decision engine**

If you want next:

I can help you **test this system with real copy** and calibrate scoring so it becomes *dangerously accurate*.